Solution: Tasks Scheduling

Problem Statement

There are ‘N’ tasks, labeled from ‘0’ to ‘N-1’. Each task can have some prerequisite tasks which need to be completed before it can be scheduled. Given the number of tasks and a list of prerequisite pairs, find out if it is possible to schedule all the tasks.

Example 1:

```
Input: Tasks=3, Prerequisites=[0, 1], [1, 2]
Output: true
Explanation: To execute task '1', task '0' needs to finish first. Similarly, task '1' needs
to finish before '2' can be scheduled. One possible scheduling of tasks is: [0, 1, 2]
```

Example 2:

```
Input: Tasks=3, Prerequisites=[0, 1], [1, 2], [2, 0]
Output: false
Explanation: The tasks have a cyclic dependency, therefore they cannot be scheduled.
```

Example 3:

```
Input: Tasks=6, Prerequisites=[2, 5], [0, 5], [0, 4], [1, 4], [3, 2], [1, 3]
Output: true
Explanation: A possible scheduling of tasks is: [0 1 4 3 2 5]
```

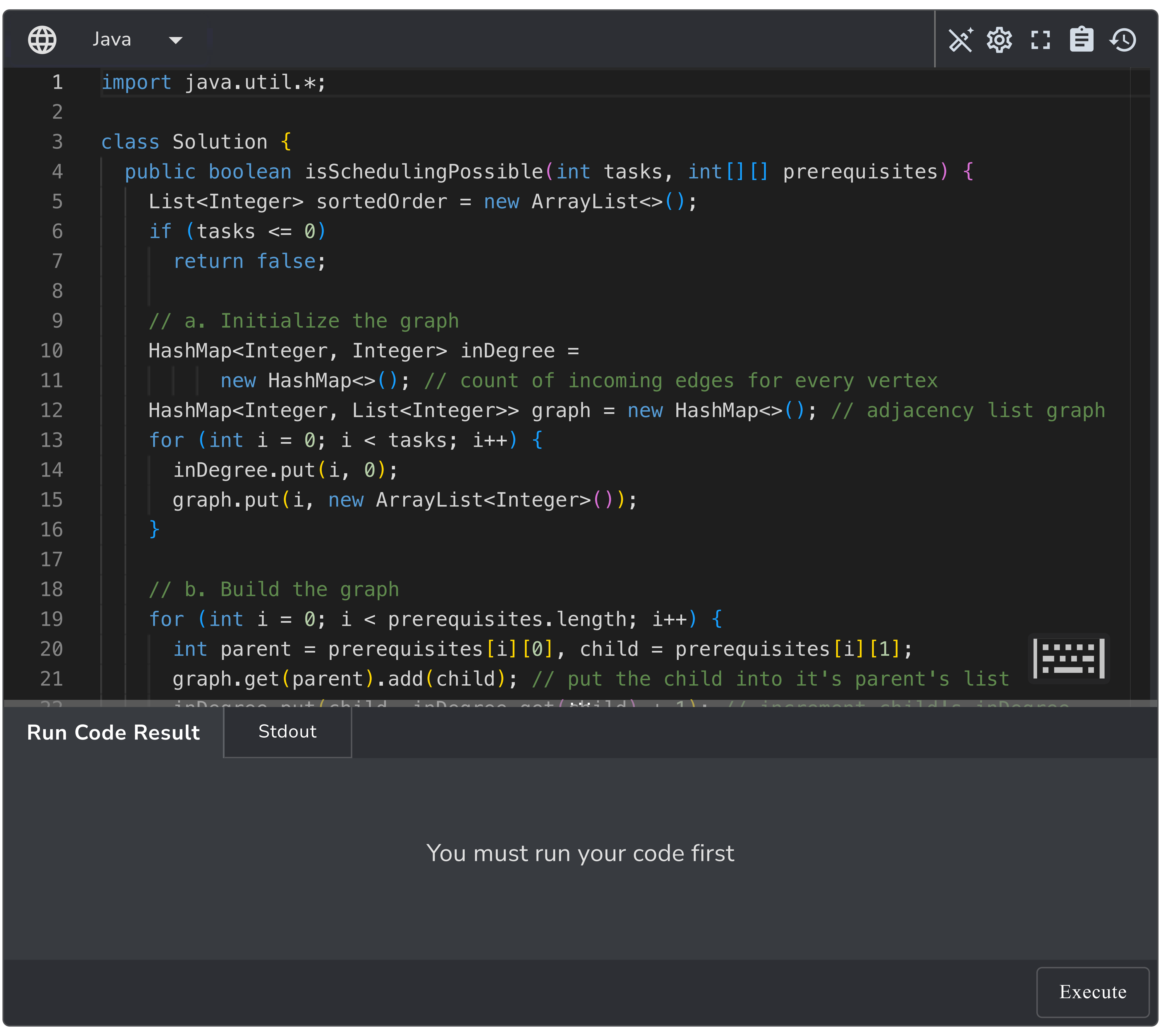
Solution

This problem is asking us to find out if it is possible to find a topological ordering of the given tasks. The tasks are equivalent to the vertices and the prerequisites are the edges.

We can use a similar algorithm as described in `Topological Sort` to find the topological ordering of the tasks. If the ordering does not include all the tasks, we will conclude that some tasks have cyclic dependencies.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {
  public boolean isSchedulingPossible(int tasks, int[][] prerequisites) {
    List<Integer> sortedOrder = new ArrayList<>();
    if (tasks <= 0)
      return false;

    // a. Initialize the graph
    HashMap<Integer, Integer> inDegree =
          new HashMap<>(); // count of incoming edges for every vertex
    HashMap<Integer, List<Integer>> graph = new HashMap<>(); // adjacency list graph
    for (int i = 0; i < tasks; i++) {
      inDegree.put(i, 0);
      graph.put(i, new ArrayList<Integer>());
    }

    // b. Build the graph
    for (int i = 0; i < prerequisites.length; i++) {
      int parent = prerequisites[i][0], child = prerequisites[i][1];
      graph.get(parent).add(child); // put the child into it's parent's list
```

Time Complexity

In step ‘d’, each task can become a source only once, and each edge (i.e., prerequisite) will be accessed and removed once. Therefore, the time complexity of the above algorithm will be O(V+E)O(V+E), where ‘V’ is the total number of tasks and ‘E’ is the total number of prerequisites.

Space Complexity

The space complexity will be O(V+E), since we are storing all of the prerequisites for each task in an adjacency list.

Similar Problems

Course Schedule: There are ‘N’ courses, labeled from ‘0’ to ‘N-1’. Each course can have some prerequisite courses which need to be completed before it can be taken. Given the number of courses and a list of prerequisite pairs, find if it is possible for a student to take all the courses.

Solution: This problem is exactly similar to our parent problem. In this problem, we have courses instead of tasks.

← Previous
Tasks Scheduling (Medium)

Next →
Tasks Scheduling Order (Medium)

☐ Mark as Completed